



COM3021 – Data Science at Scale

STORAGE AND RETRIEVAL

DR HUGO BARBOSA

Agenda

- Past

*The format in which we give the data to the database and how we query it from the **development perspective***

- Today

How databases store data and how they retrieve them back

Introduction

Databases have two fundamental jobs:

- When you give it data, it should store it
- When you request that data, it should give it back to you

Important decisions have to be made regarding storage engine having a specific workload in mind

- Type of workload

Transactional workload (write-intensive)

Analytics workload (read-intensive)

Storage engines

Storage engines

- A (very) simple database

Key-value pairs

```
db_set () {  
    echo "$1,$2" >> database  
}  
  
db_get () {  
    grep "^$1," database | sed -e "s/^$1,/" | tail -n 1  
}
```

```
db_set 25 '{"name":"London", "attractions":["Big Ben", "London Eye"]}'  
db_set 32 '{"name":"San Francisco", "attractions":["Golden Gate Bridge"]}'  
db_get 25  
{"name":"London", "attractions":["Big Ben", "London Eye"]}  
  
db_set 25 '{"name":"London", "attractions":["London Bridge"]}'  
{"name":"London", "attractions":["London Bridge"]}  
  
db_get 32  
{"name":"San Francisco", "attractions":["Golden Gate Bridge"]}
```

Storage engines

- What's the performance of our database:

Read?

Write?



Database logs

- Many databases will use *logs* to record data in the database
 - Not logs in the traditional sense*
 - Read-only data files storing the records in sequence*
 - Reading such data file will have a complexity of $O(n)$ ☹
 - We need more efficient ways to retrieve the data from logs
-

Things to consider...

- File format

Binary files are normally more efficient than plain text files

- Deleting records

Scanning the whole log to delete the record is inefficient. Instead, we normally append a special delete (tombstone) record

- Crash recovery

If the database is restarted, the hash map is lost.

- Partially-written records

- Concurrency control
-

Indexes

Hash index

- Index for key-value data, similar to dictionaries (but for disks)

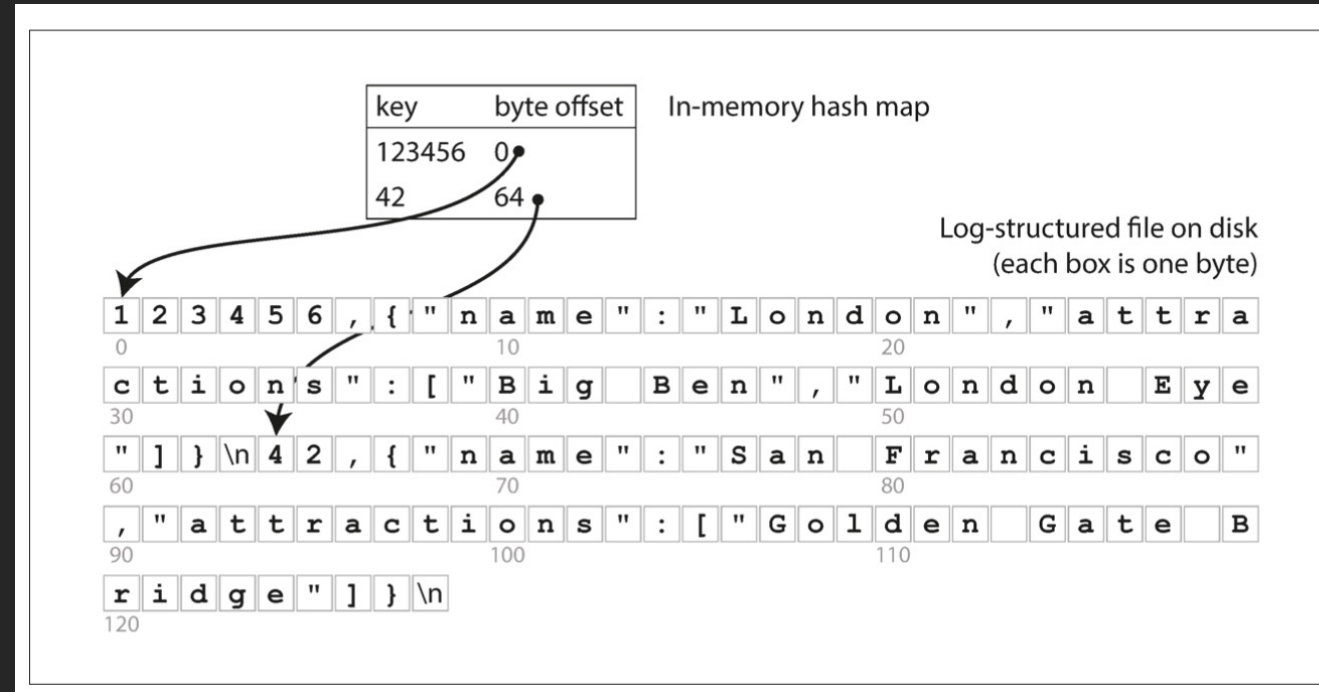
- Simple example:

Keep a hash map where every key is mapped to byte offset in the log file.

- When you append data to the file, you update the hash map

Simple but efficient

- (as long as all the indexes fit in memory)

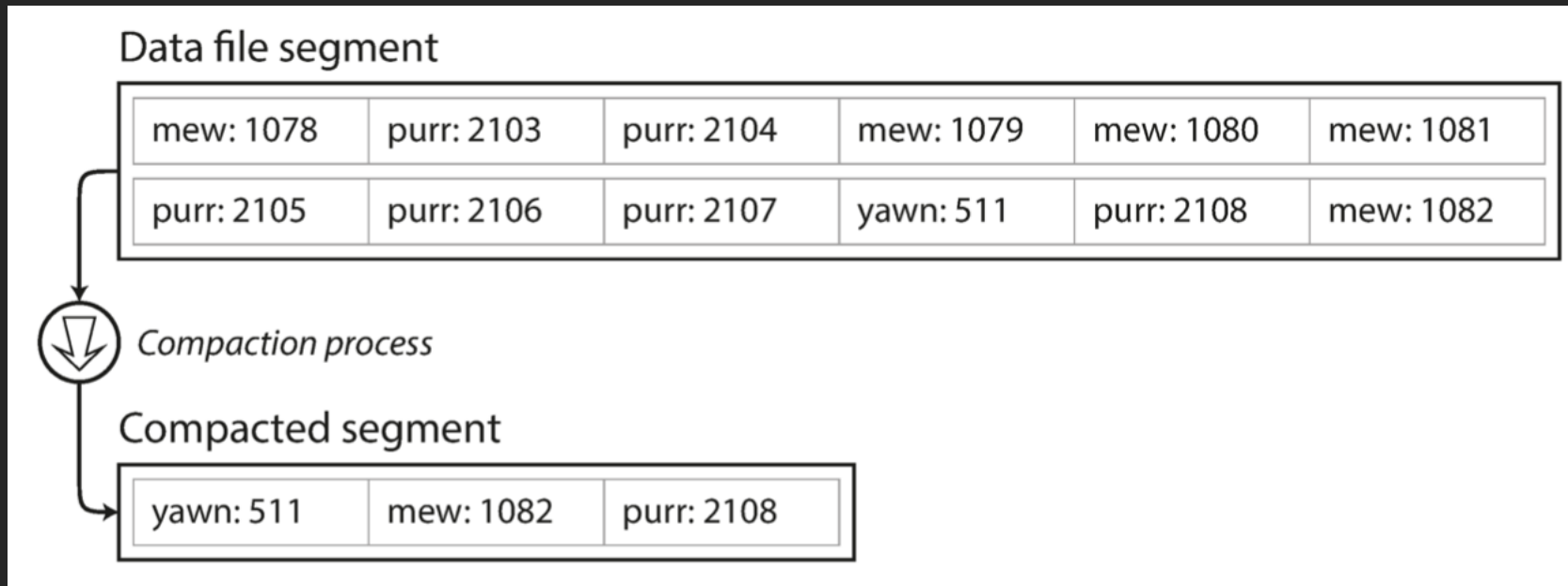


Segment files

- If we are only appending data to a file, how do we prevent running out of disk spaces?
The log is broken into segments of a certain size by closing the file when it reaches a given size.
Subsequent writes are on a new file
-

Compaction

- In order to reduce the disk usage, duplicate keys can be thrown away



Database storing the number of times each cat video was played. Lots of write operations (counter) and few keys.

Hash tables

Advantages

- *Sequential write operations are much faster than random access to disk;*
- *Crash recovery can be easier since when updating a value you're not actually overwriting its old content.*

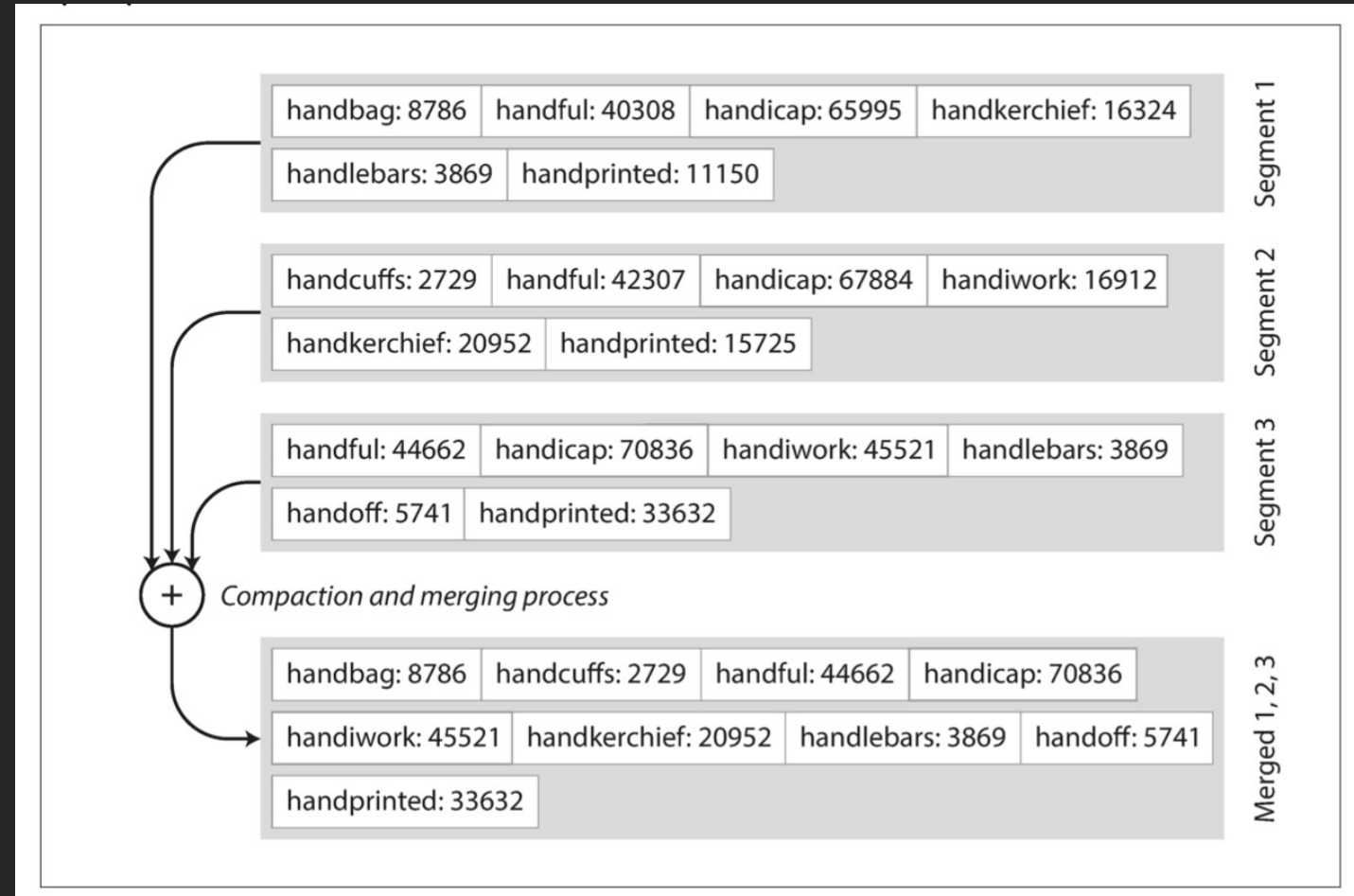
Disadvantages

- *The hash table must fit into memory*
 - *Range queries (retrieve vales for keys kitty1000 to kitty2000*
-

SSTables

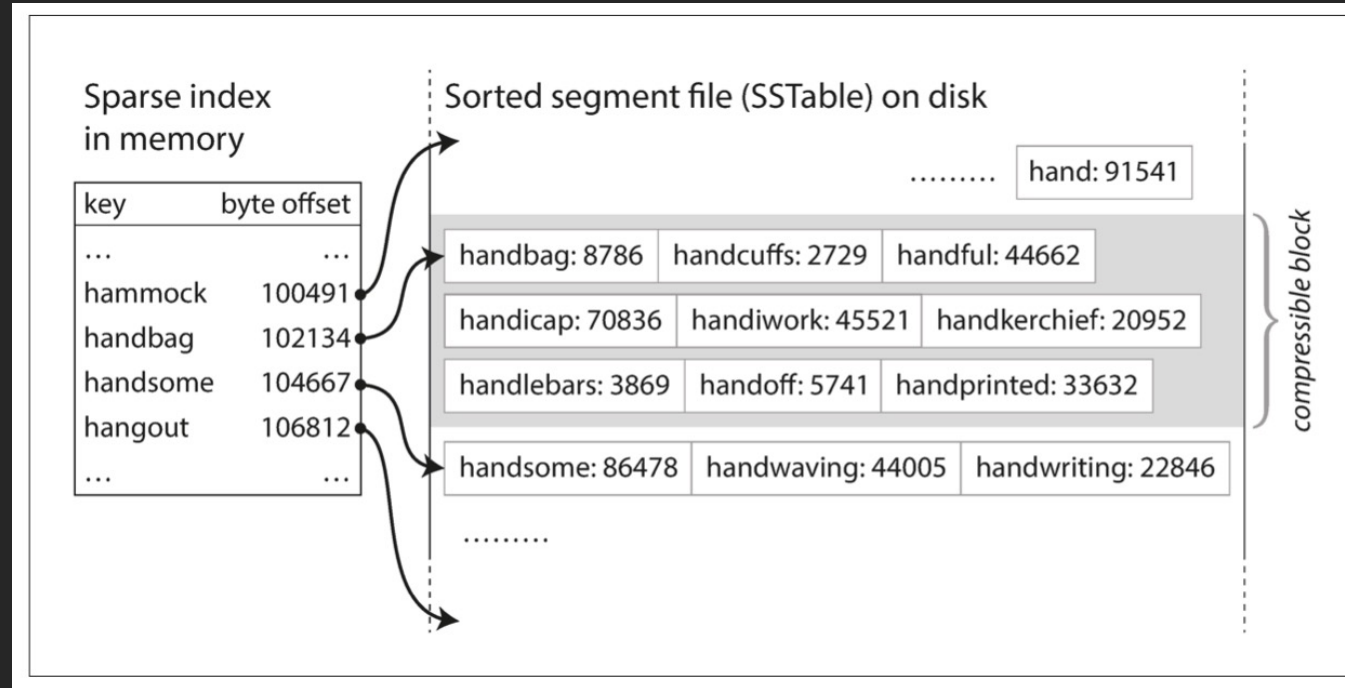
- Sorted String Tables
- Each key appears once per segment
- Keep the log sorted by key

Compaction and merging



SSTables

- Sorted String Tables
- Each key appears once per segment
- Keep the log sorted by key
- You still need an in-memory index but you don't need all the keys anymore
Offset of some of the keys



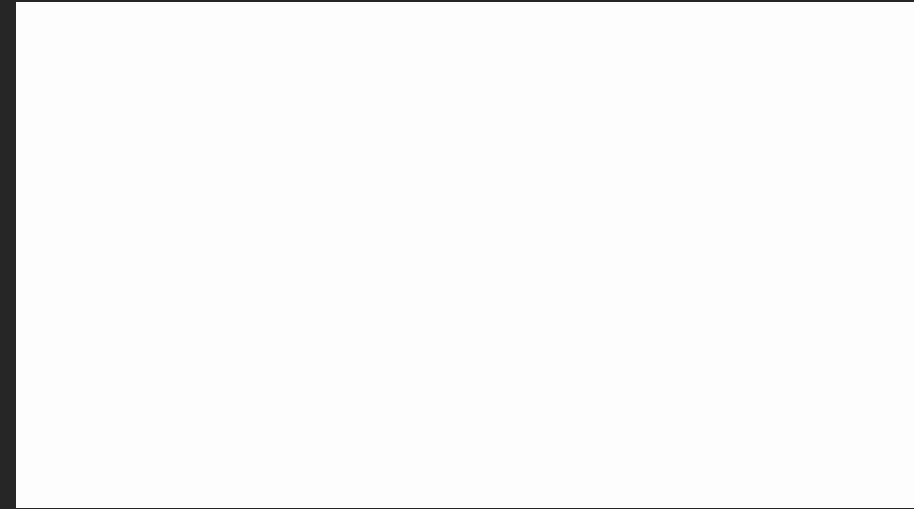
SSTables

- Sorted String Tables
- Each key appears once per segment
- Keep the log sorted by key

How do we keep it sorted?

Using a tree structure

AVL tree



SSTables

- Write

add it to an in-memory balanced tree data structure (for example, an AVL tree) (memtable)

When the memtable gets bigger, write it out to disk as an SSTable file.

The new SSTable file becomes the most recent segment of the database. While the SSTable is being written out to disk, writes can continue to a new memtable instance.

- Read:

First try to find the key in the memtable, then in the most recent on-disk segment, then in the next-older segment, etc.

From time to time, run a merging and compaction process in the background to combine segment files and to discard overwritten or deleted values.
